

RakshaSafe – AI-Powered Women Safety and Disaster Emergency Response System

CHAPTER 6 SYSTEM TESTING

B.Sc. Information Technology – Final Year Project Documentation

Developed by: Vivek & Vaibhav

This chapter presents the complete testing exercise carried out on the RakshaSafe system. It explains the testing environment, strategy and levels of testing, the types of testing performed, the structured test cases executed module by module, the results obtained, the defects that were found and fixed during testing, and the overall test summary. Every result reported in this chapter was obtained by executing the running application – the live frontend, backend and MongoDB database – with genuine test input. The chapter closes with the limitations of the current testing exercise and a brief summary of the confidence gained from it.

CHAPTER 6 – SYSTEM TESTING

6.1 Introduction

Testing is the process of executing a software system with the intention of finding errors, verifying that it behaves as specified, and gaining confidence that it will work for real users in real situations. For a safety-focused application such as RakshaSafe, testing is not merely a formality; it is the activity that decides whether the system can be trusted in the situations it is meant for. A citizen who presses the SOS button, an administrator who monitors live incidents, or a user who stores an emergency contact must be able to rely on the system behaving correctly every time.

RakshaSafe is implemented as three cooperating layers: a React + TypeScript + Vite frontend, a Node.js + Express.js + TypeScript REST backend, and a MongoDB database, with a separate Python + FastAPI service used for AI-based risk assessment. Because data flows through all these layers, testing had to happen at multiple levels – inside the code, at the API boundary, and through the complete application in a real browser. A module that works in isolation but breaks when connected to the rest of the system is not acceptable, so defects were actively hunted for rather than assumed away.

The testing effort combined structured manual test-case execution with automated checks that run in seconds: strict TypeScript compilation on both the frontend and the backend, the oxlint code-quality checker, a full production build of the frontend, programmatic API checks against the running backend, direct verification of documents inside MongoDB, and an end-to-end browser walkthrough of the main user and administrator journeys performed in Google Chrome. This mixture made the verification both repeatable and broad – details that a human tester can judge (such as layout, language and flow smoothness) were judged by eye,

while correctness details such as status codes, record creation and access control were checked mechanically wherever possible.

6.2 Objectives of Testing

The testing exercise was designed around the following clear objectives:

Verify that every functional requirement in Chapter 3 works in the running system – registration, login, profile management, emergency contacts, SOS incident creation, location handling, facilities and nearby resources, rescue teams and assignments, notifications, unsafe-area reporting, AI risk assessment, and the administrator functions.

Verify non-functional requirements, especially security and role-based access control: that unauthenticated requests are rejected, that a normal user cannot reach administrator functions, and that one user can never read or modify another user’s personal data.

Verify data integrity and ownership isolation at the database level, by inspecting the actual MongoDB documents created and by confirming that relationships reference valid records.

Verify validation of every input – required fields, phone numbers, emails, passwords, lengths and unexpected characters – so that invalid data is rejected with a clear message instead of corrupting the database.

Verify usability and responsiveness: that the interface has clear labels, works in both English and Hindi wherever provided, and adapts correctly from a desktop screen down to a small phone screen.

Verify reliability: that a successfully submitted incident persists, that the failure of optional external services (email, SMS, WhatsApp, maps, AI) never corrupts or blocks the core records, and that the application itself never crashes.

Record every defect found, fix it, and re-run the affected scenarios to prove the fix – so the final system is delivered without any known open defect.

6.3 Test Environment

All testing was performed on a Windows-based development machine running MongoDB locally. The three application layers were executed as real processes – the Vite development server on port 5173, the Express API on port 5000, and where relevant the FastAPI AI service on port 8000. The frontend was opened in Google Chrome, both as a desktop window and in the browser’s device-emulation mode so that mobile layouts could be checked without a physical phone.

Component	Environment Used
Processor / RAM	Intel Core i5-class laptop, 8 GB RAM (development baseline as per Chapter 3)
Operating system	Windows 10/11
Frontend runtime	React 19 + TypeScript 6, Vite dev server (port 5173), production build via “tsc -b && vite build”
Backend runtime	Node.js 20+ / Express 5 + TypeScript, executed with tsx (port 5000)
Database	MongoDB (local instance, mongodb://127.0.0.1:27017), Mongoose ODM
AI service	Python 3.12 + FastAPI (port 8000) – risk assessment end point
Browsers	Google Chrome (latest), including device-emulation for mobile widths
API testing tools	Command-line HTTP requests (curl) and in-browser network inspection

Static / build checks	tsc –noEmit (frontend + backend), oxlint, full Vite production build
Data verification	Direct inspection of MongoDB collections during and after each scenario

Keeping the database local and the services on the same machine made every test repeatable: the data created during a scenario could be inspected immediately in MongoDB, and any unexpected side effect could be traced to its source. Because some features – such as real SMS, email or WhatsApp delivery – depend on external providers that were not configured with credentials during testing, those paths were exercised as far as the application’s honest status recording allows, meaning the system is verified to record the delivery status (for example NOT_CONFIGURED or FAILED) without ever corrupting the underlying incident record.

6.4 Test Strategy and Levels of Testing

Testing was carried out in layers, from the smallest unit of code up to the complete running system. Each level answers a different question, and a failure at a higher level could usually be traced back to a lower one.

6.4.1 Unit Level Verification

Unit testing verifies each function in isolation, with every dependency either real (for pure utilities) or replaced, so that a failure points exactly at the code under test. A dedicated automated regression suite was written for RakshaSafe using the built-in Node.js test runner (node:test) executed through tsx, so no separate test framework is required at runtime. The suite lives under backend/tests/unit/ and covers six files:

phone.test.ts – Indian phone-number utilities: normalization to the +91 format, validity checks and display formatting.

search.test.ts – regex escaping so attacker-controlled input cannot alter a query pattern.

password.test.ts – bcrypt round-trip, wrong-password rejection and proof that only a salted hash ever reaches storage.

jwt.test.ts – signing and verification, role preservation, tamper and garbage rejection.

authValidators.test.ts – registration, login and profile-update validators, including normalization side-effects.

emergencyContactValidators.test.ts – emergency-contact create/update validators and ObjectId validation.

The table below is the complete inventory of the 41 checks that were written and executed. Every check passed; none was skipped or ignored.

ID	Suite / File	Test description	Result
U-01	phone.test.ts	Plain 10-digit number is normalized to +91	PASS
U-02	phone.test.ts	Already-prefixed +91 number is preserved unchanged	PASS
U-03	phone.test.ts	12-digit 91-prefixed number (no +) is normalized	PASS
U-04	phone.test.ts	10-digit number starting with 91 is NOT mistaken as already prefixed	PASS
U-05	phone.test.ts	Surrounding whitespace is trimmed	PASS
U-06	phone.test.ts	Common punctuation (dashes, spaces) is stripped	PASS
U-07	phone.test.ts	Too-short input is rejected	PASS
U-08	phone.test.ts	Number not starting with a valid mobile prefix (6–9) is rejected	PASS

U-09	phone.test.ts	Non-digit garbage is rejected	PASS
U-10	phone.test.ts	isValidIndianPhone: true for valid input with spaces	PASS
U-11	phone.test.ts	isValidIndianPhone: false for invalid input	PASS
U-12	phone.test.ts	formatPhoneForDisplay: formats a normalized number	PASS
U-13	phone.test.ts	formatPhoneForDisplay: returns input when unparseable	PASS
U-14	search.test.ts	Regex metacharacters escaped so input cannot alter a pattern	PASS
U-15	search.test.ts	Plain text passes through unchanged	PASS
U-16	search.test.ts	Empty and whitespace-only input handled safely	PASS
U-17	password.test.ts	Bcrypt hash verifies against the plain-text password	PASS
U-18	password.test.ts	Wrong password is rejected by comparison	PASS
U-19	password.test.ts	Identical input produces different salts; plaintext never stored	PASS
U-20	jwt.test.ts	Token verifies back to the same id and role	PASS
U-21	jwt.test.ts	Administrator token preserves the ADMIN role	PASS
U-22	jwt.test.ts	Tampered token is rejected	PASS
U-23	jwt.test.ts	Garbage string that is not a token is rejected	PASS
U-24	authValidators.test.ts	Valid registration payload accepted (email lowercased)	PASS
U-25	authValidators.test.ts	Missing name is flagged	PASS
U-26	authValidators.test.ts	Malformed email and invalid phone are both flagged	PASS
U-27	authValidators.test.ts	Password shorter than 8 characters is flagged	PASS
U-28	authValidators.test.ts	Login accepts email or phone as identifier	PASS
U-29	authValidators.test.ts	Login requires an identifier and a password	PASS
U-30	authValidators.test.ts	Profile update requires at least one field	PASS
U-31	authValidators.test.ts	Name update is trimmed before returning	PASS
U-32	authValidators.test.ts	isEmail accepts valid addresses, rejects invalid ones	PASS
U-33	authValidators.test.ts	isPhone accepts Indian format, rejects garbage	PASS
U-34	emergencyContactValidators.test.ts	Valid contact accepted; phone normalized to +91	PASS
U-35	emergencyContactValidators.test.ts	Too-short contact name is flagged	PASS
U-36	emergencyContactValidators.test.ts	Invalid Indian phone number is flagged	PASS
U-37	emergencyContactValidators.test.ts	Non-boolean alert flags are flagged	PASS
U-38	emergencyContactValidators.test.ts	Empty update (no fields) is rejected	PASS
U-39	emergencyContactValidators.test.ts	Partial phone update accepted and normalized	PASS
U-40	emergencyContactValidators.test.ts	Clearing an optional email with empty string allowed	PASS
U-41	emergencyContactValidators.test.ts	isValidObjectId accepts valid ids, rejects malformed	PASS

The checks are written as plain, human-readable assertions. A representative extract from the phone-number suite is shown below; the assertions record observable behaviour rather than implementation details:

```
import { describe, it } from 'node:test'
import assert from 'node:assert/strict'
import { normalizeIndianPhone, isValidIndianPhone } from '../src/utils/phone.js'

describe('normalizeIndianPhone', () => {
  it('normalizes a plain 10-digit Indian mobile number', () => {
    assert.equal(normalizeIndianPhone('9876543210'), '+919876543210')
  })
  it('rejects a number that does not start with a valid mobile prefix (6-9)', () => {
    assert.equal(normalizeIndianPhone('5987654321'), null)
  })
})
```

Execution and results. The suite is run with `npm run test:unit`. All 41 checks passed; the runner's summary was:

```
> npm run test:unit

✓ normalizeIndianPhone ..... 9 checks passed
✓ isValidIndianPhone ..... 2 checks passed
✓ formatPhoneForDisplay ..... 2 checks passed
```

- ✓ escapeRegExp 3 checks passed
- ✓ register/login/profile validators ... 10 checks passed
- ✓ emergency-contact validators..... 8 checks passed
- ✓ JWT signing and verification 4 checks passed
- ✓ password hashing (bcrypt) 3 checks passed

tests 41 suites 13 pass 41 fail 0 duration 46.5s

The run time is dominated by the bcrypt cost factor (12 rounds), which is deliberately high for a production safety application, so the three password checks account for most of the 46 seconds on the test laptop. The checks are deterministic and independent of the database, the network and the browser – exactly the property a regression foundation needs. If a developer later changes a normalization rule or a validator, this suite turns that change into an immediate, local pass-or-fail answer with no setup beyond npm install; being committed to the repository, it is reproducible by any examiner.

6.4.2 Integration Testing (API Level)

Unit checks prove that each function works alone; integration testing proves that the parts work together – routes reach controllers, controllers persist through Mongoose, validation gates requests, and the authentication and authorization middleware protect every endpoint. The automated integration suite (backend/tests/integration/api.integration.test.ts) drives the real Express application over real HTTP using supertest, backed by a genuine MongoDB server started in memory for the run (mongodb-memory-server downloads and runs the actual mongod binary, so the query engine and wire protocol are the production ones – only the storage location differs). Two real users and one administrator are created during the run, and the entire test database is dropped when the suite finishes, so real application code and the real database engine are exercised without touching a single byte of production data. The table below lists all twenty-one scenarios exactly as they execute, in order; note how closely it mirrors the runner output reproduced afterwards.

ID	Scenario (end-to-end through the running API)	Method / Route	Expected	Result
IT-01	Health check reports the database as connected	GET /api/health	200, db connected	PASS
IT-02	Brand-new user (User A) registers and receives a JWT	POST /api/auth/register	201 + token	PASS
IT-03	Second user (User B) registers as an isolation seed	POST /api/auth/register	201 + token	PASS
IT-04	Duplicate email registration is rejected	POST /api/auth/register	409	PASS
IT-05	Login with a wrong password is rejected generically	POST /api/auth/login	401	PASS
IT-06	Known administrator authenticates (ADMIN role)	POST /api/auth/login	200, role	PASS
IT-07	Own profile returned for the calling user	GET /api/auth/me	200, own	PASS
IT-08	Protected route without a token is rejected	GET /api/auth/me	401	PASS
IT-09	USER token on an administrator route is forbidden	GET /api/admin/dashboard	403	PASS
IT-10	Administrator route without a token is rejected	GET /api/admin/dashboard	401	PASS
IT-11	ADMIN token opens the administrator dashboard	GET /api/admin/dashboard	200	PASS
IT-12	Contact created and phone normalized to +91	POST /api/emergency-contacts	201, +91..	PASS
IT-13	Invalid contact phone number is rejected	POST /api/emergency-contacts	400	PASS
IT-14	Users list only their own contacts	GET /api/emergency-contacts	own only	PASS
IT-15	A user cannot update another user's contact	PATCH /api/emergency-contacts/:id	404	PASS
IT-16	A user can update their own contact	PATCH /api/emergency-contacts/:id	200, updated	PASS
IT-17	Malformed ObjectId rejected cleanly, not a crash	PATCH /api/emergency-contacts/:id	400	PASS
IT-18	Updating a non-existent contact returns 404	PATCH /api/emergency-	404	PASS

		contacts/:id		
IT-19	Administrator lists all contacts with owner info	GET /api/admin/emergency-contacts	200, owner	PASS
IT-20	Administrator updates and then deletes any contact	PATCH + DELETE admin :id	200, 200	PASS
IT-21	Administrative actions recorded in the admin log	AdminLog collection	count > 0	PASS

Three of these scenarios read as follows in the actual test source – registration, the role gate, and ownership isolation:

```
// IT-02 a brand-new user registers and receives a JWT
const res = await request(app).post('/api/auth/register').send({
  name: 'Integration User A', email: 'it.user.a@example.com',
  phone: '9876543210', password: 'Password@123', language: 'en',
})
assert.equal(res.status, 201)
assert.ok(res.body.data.token)

// IT-09 a USER token is forbidden from administrator routes
const res2 = await request(app).get('/api/admin/dashboard')
  .set('Authorization', `Bearer ${userAToken}`)
assert.equal(res2.status, 403)

// IT-14 user B can never see user A's contacts
const other = await request(app).get('/api/emergency-contacts')
  .set('Authorization', `Bearer ${userBToken}`)
assert.equal(other.body.data.contacts.length, 0)
```

The full run is reproduced below – the runner reported all 22 passing results (twenty-one scenarios plus the outer test block), with zero failures:

```
> npm run test:integration

✓ health check reports the database as connected
✓ a brand-new user can register and receives a JWT
✓ a second user cannot see the first user's data
✓ duplicate email registration is rejected
✓ login rejects a wrong password with a generic message
✓ a known administrator can authenticate
✓ GET /api/auth/me returns the authenticated profile
✓ unauthenticated access to a protected route is rejected (401)
✓ a USER token is forbidden from administrator routes (403)
✓ missing auth on an administrator route is rejected (401)
✓ an ADMIN token can open the administrator dashboard (200)
✓ user creates an emergency contact and it is normalized and stored
✓ creating a contact with an invalid phone number is rejected (400)
✓ a user lists only their own contacts (ownership isolation)
✓ a user cannot update another user's contact
✓ a user can update their own contact
✓ a malformed ObjectId in a path is rejected (400), not a crash
✓ updating a non-existent contact returns 404
✓ an administrator can list all emergency contacts with owner info
✓ an administrator can update and then delete any contact
✓ admin activity is recorded in the admin logs

tests 22   suites 0   pass 22   fail 0   duration 13.0s
```

Two results deserve emphasis. First, a wrong password produces the same generic 401 body as an unknown account, so the login endpoint cannot be used to enumerate registered users. Second, ownership isolation came back clean in every attempt: user B could never read, update or delete a record belonging to user A, and the administrator cross-user path succeeded only with an ADMIN token. These automated results repeat the findings of the

manual API phase, during which every endpoint was also exercised against the running backend and the resulting documents were inspected directly in MongoDB after each operation. The role-based access-control matrix in Section 6.5.2 summarises both phases.

6.4.3 System Testing

At the top of the test pyramid, the complete application – frontend, backend and database working together – was tested as one product in a real browser, on the environment described in Section 6.3. Every page was navigated, every form submitted, and every screen state (loading, empty, error, success) observed. This is the level where cross-cutting problems appear: an API that returns data a page cannot render, a redirect that sends a user to the wrong place, or a message that only exists in one language. The table below lists the twelve areas exercised end to end; the module-by-module details and all seventy system-level test cases, each with its scripted steps and observed result, are tabulated in Section 6.7.

#	Test area (exercised end to end in the browser)	Key checks performed	Result
S-01	Authentication and authorization	sign-up, sign-in, sign-out, role-based redirects, 401 / 403 behaviour	PASS
S-02	User and profile management	view and update profile, language switch (English / Hindi)	PASS
S-03	Emergency contacts	add, edit, delete, mark primary, validation messages	PASS
S-04	SOS and incident management	one-tap SOS, incident confirmation, history and incident detail	PASS
S-05	Location handling	location permitted, denied and unavailable – no fabricated coordinates	PASS
S-06	Facilities and nearby resources	list, search and filter of nearby facilities	PASS
S-07	Rescue teams and assignments	team list, assignment to incidents, status flow	PASS
S-08	Notifications	notification list, unread states, external-provider status recording	PASS
S-09	Unsafe-area reporting	report submission and visibility in map / feed	PASS
S-10	AI risk assessment	risk-analysis end point availability, output structure and score range	PASS
S-11	Administrator functions	dashboard, users, facilities, teams, incidents, reports, admin logs	PASS
S-12	Validation, edge cases, data integrity	primary-contact uniqueness, persistence after reload	PASS

Cross-cutting observations recorded during these system-level runs:

Navigation consistency – the SOS flow is reachable in one tap from every user screen, and the administrator reaches every management page from a single sidebar.

Form behaviour – required fields are clearly marked, validation errors appear next to the offending field in the user’s selected language, and repeated bad submissions never crash the page.

Data persistence – every successful write survived a browser reload and reappeared in both the user’s and the administrator’s views.

Browser console – no uncaught JavaScript exception or failed network request was observed during the walkthrough; the frontend also compiles into a clean production build.

Responsiveness – desktop, tablet (768 px) and mobile (approximately 375 px) layouts were checked; narrower screens switch from tables to stacked cards without cutting off content.

The screenshots captured during these runs are stored in the repository under `documentation/test-evidence/screenshots/` as permanent evidence of the results quoted in this chapter, and the black-and-white system diagrams used in Chapter 4 are archived under `documentation/ch4_bw/`. All seventy system-level test cases in Section 6.7 passed.

6.4.4 User Acceptance Testing

An end-to-end acceptance walkthrough of the complete user and administrator journey was executed in Google Chrome: sign-up, sign-in, emergency contact management, SOS incident creation, notification checking, the administrator dashboard, and role-based redirects. All nine steps of the walkthrough passed without any console error in the browser.

6.4.5 Regression Testing

After every defect fix the previously executed core scenarios were re-run to prove that the fix had not broken anything else. The most important example is the account access defect described in Section 6.8: after the fix, the complete acceptance walkthrough was executed again and passed end to end. Regression is now also automated: the 41 unit checks and 22 integration checks described in Sections 6.4.1 and 6.4.2 are re-run in under a minute after any code change (npm test), so a future change that breaks existing behaviour is caught immediately, not only during a manual pass.

6.5 Types of Testing Performed

6.5.1 Functional Testing

Functional testing is where the product meets the promises made in Chapter 3. Every functional requirement was converted into an executable scenario and then actually performed against the running application – not assumed, not eyeballed, performed. Each scenario asked a deliberately plain question: does this screen do what the requirement says it should, and is the data it handles stored and shown back correctly? For almost every feature, two versions of the scenario were run. The first was the happy path, exactly how a normal person would use the feature – registering, adding a contact, pressing SOS, opening a report. The second was the stubborn path: submitting blank fields, a phone number with a mistyped digit, a far-too-short password, or an email without an @ symbol, each time confirming that a clear validation message appeared next to the offending field and that no invalid record ever reached the database. The seventy cases in Section 6.7 document this combination in full, module by module, with the steps performed and the result actually observed. As an integrity check on the build itself, the frontend was also compiled into a production build (TypeScript project build followed by a Vite production build) and the backend verified to type-check cleanly, so the code that was functionally exercised is the same code that would be deployed.

6.5.2 Security and Role-Based Access Control Testing

For a women-safety product, security is not one feature among many; it is the reason the product exists, and it was treated accordingly. The back end enforces three gates, each of which must hold independently: authentication (a signed JWT must be presented), role

authorization (administrator-only routes must refuse non-admin identities), and data ownership (a request may only reach the caller's records). Rather than trust that these gates exist, each one was tested with three kinds of request – no token at all, a normal-user token, and an administrator token – and the returned status codes were checked against the expectation: 401 when authentication was missing or invalid, 403 when a valid user reached for an administrator function, and 200 when the caller was genuinely allowed. The matrix below records what actually came back.

Endpoint (area)	No token	USER token	ADMIN token
POST /api/auth/login, GET /api/auth/me	401	200 (self)	200 (self)
/api/emergency-contacts (own contacts)	401	200 (own only)	200
/api/admin/dashboard	401	403	200
/api/admin/users	401	403	200
/api/admin/emergency-contacts (all contacts)	401	403	200
/api/admin/facilities, /api/admin/teams	401	403	200
/api/admin/incidents, assignments, reports	401	403	200
/api/admin/.../:id of another user's record	401	403	200 or 404

Beyond the matrix, ownership isolation was probed directly rather than only at route level. With records belonging to several users present in the database, a normal user consistently received only their own emergency contacts, incidents and notifications; attempts to update or delete a record using another user's identifier were refused, whether the identifier was guessed or copied from a page. Two supporting details are worth recording because they are easy to get wrong: passwords exist in storage only as salted bcrypt hashes and never in plain text, and the login end point returns the same generic message for a wrong password as for an unknown account, so an outsider cannot use it to confirm which users are registered.

6.5.3 Usability Testing

Usability was reviewed the way a safety product must be judged – through the eyes of a person who may be stressed or in a hurry. Every page was loaded as a first-time visitor and asked three questions: is the purpose clear from the title, are the fields labelled properly, and can I recover when I make a mistake? Required fields are marked, and a validation error appears beside the field that caused it, in the language the user selected rather than a browser default. The most safety-critical action – the SOS flow – is reachable in one tap from the navigation on every user screen, so it never has to be hunted for. The English and Hindi language selector was toggled throughout the walkthrough, and every screen carrying placeholders or messages was read in both languages to confirm that nothing had been left untranslated or awkwardly truncated.

6.5.4 Compatibility and Responsiveness Testing

A safety application lives in the pocket, so desktop-only coverage would have been dishonest. Every main page was opened at desktop width and then in the browser's device-emulation mode at tablet (768 px) and mobile (approximately 375 px) widths. The layouts adapt as designed: wide screens present records in tabular rows, and narrower screens quietly switch to stacked cards so that no information is cut off and no button slides out of reach. Because the implementation is standards-based, behaviour is expected to remain consistent

across modern browsers; the current exercise verified it hands-on in Google Chrome at those three widths, and other browsers are recorded in Section 6.10 as not separately exercised.

6.5.5 Reliability and Data Integrity Testing

A system that forgets its own records is as dangerous as one that never had them, so the integrity tests were deliberately unkind to real data. An SOS incident was submitted, confirmed to persist in MongoDB immediately, and then re-opened from both the user's history and the administrator's incident list after a browser reload. An incident created while location was unavailable was saved without fabricating a fallback position – the location field was left genuinely absent, because a guessed coordinate sent to rescue teams would be worse than none. Booking-style rules were verified at the database level too: a user may mark exactly one primary emergency contact, and that rule is enforced by a partial unique index in MongoDB rather than trusted to the screen alone. Administrative activity was likewise confirmed to persist in the admin-log collection, so the audit trail survives the transaction that created it.

6.5.6 Performance Testing

Performance was observed on the local development machine with the same running system used for every other test in this chapter. The integration runs timed each operation, and the genuinely measured latencies of representative requests are listed below:

Operation (measured during the integration run)	Observed latency
Health check (GET /api/health)	about 0.1 s
User profile (GET /api/auth/me)	about 0.04 s
Create emergency contact (POST)	about 0.02 s
List own contacts (GET)	about 0.05 s
List all contacts, administrator (GET)	about 0.06 s
Update and delete a contact, administrator	about 0.07 s
Login (includes bcrypt comparison)	about 0.8 s
Register new account (includes bcrypt hashing)	about 1.3 s

Almost every operation completes in well under a tenth of a second. The two noticeably slower requests are login and registration, and there is a deliberate reason: both carry the bcrypt cost factor of twelve rounds, the price paid for keeping password hashes resistant to cracking – a consciously sound trade for a safety product. Page navigation, dashboard loading and SOS submission stayed within the same comfortable range during the browser runs. Formal load and soak testing with large numbers of concurrent users was not part of this exercise; that boundary is recorded honestly in Section 6.10.

6.6 Test Data Preparation

All test data used in this chapter was genuine data created in the running system through the registration screen, the API, or dedicated seed scripts – nothing was hard-coded into the application to make screens look filled. Several dedicated accounts were registered covering all three roles: normal users (including two users used specifically to prove that one user cannot see another's data), an administrator, and a responder. Test phone numbers used reserved Indian mobile-format numbers, and all email addresses were synthetic test addresses. Records (contacts, incidents, facilities, rescue teams, assignments, unsafe-area

reports) were created, updated and deleted during testing, and the database was inspected after each scenario to confirm exactly what had been written. This approach means every test result below describes behaviour that was actually observed on real, running software.

6.7 Test Cases and Results (Module by Module)

The test cases below were executed against the running system. Each case gives a scripted description, the steps performed, the expected result, the result actually observed, and the final status. All cases listed were executed; a defect discovered in an earlier run (Section 6.8) was fixed and the corresponding cases were re-executed and passed. The abbreviation “ERR” in a step means the test expects the system to reject the input and show a clear message.

6.7.1 Authentication and Authorization

The authentication module is the gateway to the whole system. It was tested for correct registration, correct sign-in, token issuance, and the rejection of invalid attempts.

TC ID	Test Description	Test Steps	Expected Result	Actual Result	Status
TC-01	User registration with valid details	Open /register; enter name, valid email, phone, password; submit	Account created; redirected to dashboard	PASS	PASS
TC-02	Duplicate email registration	Register again with the same email	Rejected with a clear validation message	PASS	PASS
TC-03	Login with correct credentials	Enter registered email and password; submit	JWT issued; redirected to role home	PASS	PASS
TC-04	Login with wrong password	Login with correct email but wrong password	401 “Invalid credentials.”; no details leaked	PASS	PASS
TC-05	Login with unknown email	Login with an unregistered email	401 “Invalid credentials.”	PASS	PASS
TC-06	Protected API without token	Request /api/auth/me with no Authorization header	401 Unauthorized	PASS	PASS
TC-07	USER on admin API	Use a normal-user JWT on /api/admin/dashboard	403 Forbidden	PASS	PASS
TC-08	ADMIN on admin API	Use an administrator JWT on /api/admin/dashboard	200 OK with dashboard data	PASS	PASS
TC-09	Logout	Click Logout from the header	Token cleared; protected pages redirect to /login	PASS	PASS

6.7.2 User and Profile Management

Profile management was verified for viewing and updating profile details and preferring a language.

TC ID	Test Description	Test Steps	Expected Result	Actual Result	Status
TC-10	View profile	Open the Profile page	Name, phone, email and role shown correctly	PASS	PASS
TC-11	Update display name and language	Change name and language (English/Hindi); save	Saved; interface labels switch language	PASS	PASS
TC-	Empty required	Attempt to save a blank name	ERR; validation	PASS	PASS

12	profile field		message shown		
----	---------------	--	---------------	--	--

6.7.3 Emergency Contacts

Emergency contacts are core to the SOS notification path. A user must manage only their own contacts, and alert settings must persist.

TC ID	Test Description	Test Steps	Expected Result	Actual Result	Status
TC-13	Add an emergency contact	Open Contacts; enter name, phone, relation; enable SMS alert; save	Contact listed with badges	PASS	PASS
TC-14	Add contact with optional email	Repeat TC-13 with an email also entered	Email stored and shown	PASS	PASS
TC-15	Own contacts only	Compare two users' contact lists	Each user sees only their own contacts	PASS	PASS
TC-16	Edit a contact	Change phone and relation of a saved contact; save	Changes reflected; updatedAt changes	PASS	PASS
TC-17	Delete a contact	Delete an existing contact	Removed from list and from MongoDB	PASS	PASS
TC-18	Single primary contact	Mark two contacts as primary	Second attempt rejected; only one primary allowed	PASS	PASS
TC-19	Invalid phone number	Enter a non-Indian-format phone	ERR; clear validation message	PASS	PASS
TC-20	Admin sees all contacts	Open Admin – Emergency Contacts	All users' contacts listed with owner name/phone	PASS	PASS
TC-21	Admin edits/deletes a contact	Edit the phone; then delete the same contact	Change visible to the owner; deletion removes the record	PASS	PASS

6.7.4 SOS and Incident Management

The incident module is the heart of the system. It was verified for creation, persistence, history, status workflow and visibility rules.

TC ID	Test Description	Test Steps	Expected Result	Actual Result	Status
TC-22	Create SOS incident	Open SOS; pick incident type/category; enter description; submit	Incident created with status REPORTED	PASS	PASS
TC-23	Incident persists immediately	Refresh the list after submission	Incident present; exists in MongoDB	PASS	PASS
TC-24	Incident saved without location	Create SOS while geolocation is denied	Created with no false location	PASS	PASS
TC-25	Incident history	Change status through the workflow	IncidentUpdate records appended in order	PASS	PASS
TC-26	Status workflow	Move REPORTED -> ACKNOWLEDGED -> ASSIGNED -> IN_PROGRESS -> RESOLVED -> CLOSED	Each transition accepted and recorded	PASS	PASS

TC-27	Cancel an incident	Cancel an open incident	Status becomes CANCELLED with a history entry	PASS	PASS
TC-28	User sees only own incidents	Open incident list as a normal user	Only own incidents shown	PASS	PASS
TC-29	Admin sees all incidents	Open Admin incidents with details	All incidents visible; filters work	PASS	PASS
TC-30	Empty/invalid description	Submit SOS with a blank description	ERR; validation message; nothing saved	PASS	PASS

6.7.5 Location Handling

Location is captured from the browser only with the user's permission, and its absence is handled honestly.

TC ID	Test Description	Test Steps	Expected Result	Actual Result	Status
TC-31	Capture location on grant	Submit an incident while allowing geolocation	Latitude/longitude stored with the incident	PASS	PASS
TC-32	Location denied gracefully	Deny geolocation and submit	No coordinates stored; flow completes	PASS	PASS
TC-33	Location accuracy recorded	Inspect a saved location document	Accuracy value stored when provided	PASS	PASS

6.7.6 Facilities and Nearby Resources

Resource discovery was verified for the user and CRUD management for the administrator.

TC ID	Test Description	Test Steps	Expected Result	Actual Result	Status
TC-34	List facilities	Open Resources; apply no filter	Hospitals/shelters/others listed	PASS	PASS
TC-35	Search/filter resources	Type a keyword and filter by type	Matching results only	PASS	PASS
TC-36	Nearby resources	Request nearby facilities using a test location	Ordered list or graceful "unavailable" message	PASS	PASS
TC-37	Admin add/edit facility	Add a facility; edit its phone and status	Saved; visible to users	PASS	PASS
TC-38	Admin disable a facility	Set a facility to not operational	Marked in listing; not offered as active	PASS	PASS

6.7.7 Rescue Teams and Assignments

Team records and software-record assignments were verified, with administrator-only access.

TC ID	Test Description	Test Steps	Expected Result	Actual Result	Status
TC-39	Admin register/add a team	Add a rescue team with type and phone	Team listed in Admin Teams	PASS	PASS

TC-40	USER cannot manage teams	Attempt team management with a USER JWT	403 Forbidden	PASS	PASS
TC-41	Create an assignment	Assign a team to an open incident	Assignment recorded with status ASSIGNED	PASS	PASS
TC-42	Assignment lifecycle	Move ASSIGNED -> EN_ROUTE -> ON_SCENE -> COMPLETED	Each transition recorded	PASS	PASS
TC-43	Invalid incident/team id	Assign using a malformed ObjectId	400 validation error	PASS	PASS

6.7.8 Notifications

Notification behaviour was tested both with real in-app delivery and with external channels honestly reported when unconfigured.

TC ID	Test Description	Test Steps	Expected Result	Actual Result	Status
TC-44	In-app notification on incident event	Create an incident; open Notifications	New in-app notification appears	PASS	PASS
TC-45	Unread count	Refresh after an unread notification	Bell badge shows the unread count	PASS	PASS
TC-46	Contact notified via SMS if configured	Create incident with SMS-enabled primary contact	Notification recorded; SENT/DELIVERED or FAILED per provider	PASS	PASS
TC-47	Unconfigured channel handled honestly	Trigger a channel with no provider set	Status NOT_CONFIGURED/UNAVAILABLE; incident unaffected	PASS	PASS

6.7.9 Unsafe-Area Reporting

Reports are moderated: submission is open, but confirmation happens only after review.

TC ID	Test Description	Test Steps	Expected Result	Actual Result	Status
TC-48	Submit unsafe-area report	Submit a report with location, category, description	Record created; marked not-verified	PASS	PASS
TC-49	Admin review and verify	Open Admin Unsafe Reports; verify the report	isVerified becomes true; visible as verified	PASS	PASS
TC-50	Not auto-confirmed	Check a fresh report without review	Not treated as confirmed until verified	PASS	PASS

6.7.10 AI Risk Assessment

The risk assessment returns decision-support scores whose structure and range were verified.

TC ID	Test Description	Test Steps	Expected Result	Actual Result	Status
TC-	Risk score in	Trigger assessment for a test	Score 0–100	PASS	PASS

51	range	location	returned		
TC-52	Risk level mapping	Check the level returned with the score	LOW/MEDIUM/HIGH/CRITICAL mapped consistently	PASS	PASS
TC-53	Service unavailable gracefully	Stop the AI service; request an assessment	Clear message; core app unaffected	PASS	PASS

6.7.11 Admin Functions – Dashboard, Users, Reports, Logs

Administrator functions were verified for correctness against real stored data and strict role gating.

TC ID	Test Description	Test Steps	Expected Result	Actual Result	Status
TC-54	Dashboard statistics	Open Admin Dashboard	Counts match the actual MongoDB data	PASS	PASS
TC-55	User listing with search	Search users by name/phone; page through results	Correct filtered paginated results	PASS	PASS
TC-56	User detail with incidents/contacts	Open a user's detail page	Only that user's related records shown	PASS	PASS
TC-57	Analytics/reports view	Open Admin Reports	Statistics derived from stored data	PASS	PASS
TC-58	Report export	Generate a report and export	PDF/CSV/JSON produced from data snapshot	PASS	PASS
TC-59	Admin activity logging	Perform several admin actions	AdminLogs entries created with action/target/time	PASS	PASS
TC-60	Logs restricted to admins	Request admin logs with a USER token	403 Forbidden	PASS	PASS
TC-61	Admin Emergency Contacts page	Open Admin Emergency Contacts; search	All contacts with owners listed; search works	PASS	PASS

6.7.12 Validation, Edge Cases and Data Integrity

A final round of cases attacked the system with malformed and unusual input.

TC ID	Test Description	Test Steps	Expected Result	Actual Result	Status
TC-62	Empty required fields	Submit each main form with fields left blank	ERR on every required field; nothing persisted	PASS	PASS
TC-63	Whitespace-only input	Submit fields containing only spaces	Treated as empty and rejected	PASS	PASS
TC-64	Special characters in search	Search with characters such as () [] * +	Returned results or empty list; no error/crash	PASS	PASS
TC-65	Very long input	Enter text well beyond field limits	Rejected with a length-validation message	PASS	PASS
TC-66	Malformed ObjectId	Use an invalid id in a path parameter	400 validation error, not a crash	PASS	PASS
TC-	Non-existent	Update/delete a valid-shaped	404 not found	PASS	PASS

67	record id	id that does not exist			
TC-68	Phone normalization	Enter 10-digit, +91 and spaces variants	Normalized to the same valid form	PASS	PASS
TC-69	Unicode / Hindi input	Save a name written in Hindi	Stored and displayed correctly	PASS	PASS
TC-70	Ownership isolation (cross-user)	Attempt to read/delete another user's contact via its id	Rejected; target data not exposed	PASS	PASS

6.8 Defects Found and Fixed During Testing

Testing is valuable only when it actually finds problems. Two genuine defects were found during this exercise. Both were fixed, and the affected scenarios were re-executed successfully. They are documented below with their root cause and the verification of the fix, which is the pattern followed whenever a defect is reported.

Defect 1 – Valid administrator account appeared “deactivated”

Symptom: A legitimate administrator whose account was created directly in the database could not log in (the API returned “Invalid credentials.”) and every administrator API request returned “Account is deactivated.” The account’s credentials were correct, yet the application was sending the user back to the login page every time.

Root cause: The account document had been inserted without the `isActive` field. The login and authorization logic checked “is the account deactivated?” using a strict comparison that treated a missing value as deactivated. Because the boolean field was undefined on this record, a perfectly active administrator was classified as disabled.

Fix: The deactivation checks in the login controller and in the authorization middleware were corrected to treat a record as deactivated only when the field is explicitly false. This preserves the behaviour for accounts that are genuinely switched off while restoring access for active records that simply predate the field.

Verification: The administrator was then able to authenticate, and the previously failing endpoints (`/api/auth/me`, `/api/admin/dashboard`, `/api/admin/users`) all returned 200. The full browser acceptance walkthrough was re-run and passed all nine steps, confirming no regression.

Defect 2 – Emergency-contacts administration page routed to login

Symptom: Navigating to the administrator’s Emergency Contacts destination fell back to the login page because the client-side hash router did not recognise the route, so no page component was rendered.

Root cause: The page, its route entry in the hash router and the navigation menu item had not been wired into the front end, even though the backend endpoints supporting the page already existed.

Fix: The admin Emergency Contacts page was implemented, the route was added to the typed route list and the known-routes table, and a navigation entry was added to the administrator menu. The page fetches all emergency contacts with the owner’s name, phone and email, supports search and pagination, offers a call (tel:) action, and allows the administrator to edit or delete any contact through the protected admin API.

Verification: With an administrator token the endpoint returned 200 with all contacts; with a normal-user token it returned 403, and without a token 401. Editing a contact from the admin page was confirmed to reflect for the owning user, and deleting it removed the real record from MongoDB.

Separately, minor hardening was added during testing: search text is escaped before building regular expressions, so special characters cannot break queries or be abused as injection patterns. This was verified with a dedicated case (TC-64).

6.9 Test Summary

A total of seventy test cases were designed across the twelve areas described above, covering the full functional scope of the system together with security, validation and edge-case behaviour. Every case was executed against the running application. The summary below shows the distribution and the overall pass rate.

Test Area	Cases Executed	Passed	Failed
Authentication and authorization	9	9	0
User and profile management	3	3	0
Emergency contacts	9	9	0
SOS and incident management	9	9	0
Location handling	3	3	0
Facilities and nearby resources	5	5	0
Rescue teams and assignments	5	5	0
Notifications	4	4	0
Unsafe-area reporting	3	3	0
AI risk assessment	3	3	0
Admin functions (dashboard/users/reports/logs)	8	8	0
Validation, edge cases and data integrity	9	9	0
Total	70	70	0

Overall pass rate: 100% (70 of 70 cases). The two defects discovered during the exercise were resolved and their areas re-tested, which is why the final totals show no open failures. These results cover the functional, security, usability, compatibility, reliability and basic performance dimensions of the system on the documented test environment.

In addition to the seventy manually scripted scenarios above, the automated regression suite described in Sections 6.4.1 and 6.4.2 contributes a further 63 repeatable checks – 41 at unit level and 22 across the integration/API level. The combined tally of every verification in this chapter is therefore 133 checks, all passing:

Level of Testing	Mechanism	Checks	Result
Unit (code utilities and validators)	node:test + tsx (automated)	41	41 passed
Integration / API (real HTTP + real MongoDB)	supertest + in-memory mongod (automated)	22	22 passed
System (frontend + backend + database in browser)	manual scripted walkthroughs	70	70 passed
Total		133	133 passed (100%)

The automated checks are re-run as a single command (npm test) and are part of the repository, so the confidence they provide is repeatable and verifiable by the examiner.

6.10 Limitations of the Testing Exercise

It is important to state what this testing exercise did not cover, so that the confidence it provides is not overstated:

All testing ran on a single local development machine with MongoDB installed locally. No multi-machine, staging or production environment was exercised.

External communication providers (real SMS, real email, WhatsApp) were not configured with live credentials. Their failure paths were verified through the application's status-recorded behaviour (NOT_CONFIGURED / FAILED / UNAVAILABLE), but real end-to-end message delivery was not measured.

Performance was observed on local hardware with light concurrent use; formal load testing, soak testing and benchmarking at scale were not performed.

The AI risk assessment was verified for service availability, output structure and score range; its analytical accuracy is decision-support software and was not validated against ground truth.

Browser coverage was concentrated on Google Chrome at desktop, tablet and mobile emulation widths; other browsers were not separately exercised.

The automated regression suite (unit + integration) is committed to the repository and runs with npm test against an in-memory MongoDB server, so it never touches production data. It is not yet wired into an external continuous-integration service; on this project it runs on the developer's machine as part of every change cycle.

6.11 Chapter Summary and Conformance

This chapter has described a complete, evidence-producing testing exercise for RakshaSafe. The system was tested at the code, API and full-browser levels against the requirements defined in Chapter 3. Sixty-three automated checks (41 unit and 22 integration) exercise the fastest-changing core – validation, hashing, tokens, authentication and ownership isolation – on every code change, while seventy system-level scenarios covered the complete functional scope in a real browser. Security was given special attention: authentication, role-based authorization and ownership isolation were verified with explicit 401 / 403 / 200 checks, and a normal user was repeatedly shown to be unable to reach administrator functions or another user's data.

All 133 checks passed on the documented environment, and every defect found during testing was fixed and then re-verified. The remaining limitations are clearly recorded in Section 6.10 and are consistent with the scale of the project and its development environment.

In conclusion, the system satisfies its verified requirements within the scope of this testing exercise:

TypeScript builds pass cleanly for both frontend and backend, and the frontend produces a working production build.

All functional modules operate correctly with genuine test data stored in MongoDB.

The automated regression suite – 41 unit checks and 22 integration checks – passes with zero failures and never touches production data.

Role-based access control and ownership isolation are enforced across the API, with no unauthorized access observed.

Validation rejects malformed input everywhere it was tested, and no test caused a crash or data corruption.

The end-to-end user and administrator journey passed in a real browser without any console error.

The two defects found were resolved and their areas re-tested successfully.